

GNOT: Generative Node Orchestration Technology

A Minimal-Seed Architecture for LLM-Native Distributed Execution

Quoc Viet Tran

gnot.io

viettran@gnot.io

March 2026

Abstract

We present **Generative Node Orchestration Technology (GNOT)**, a distributed execution mesh designed to be orchestrated natively by large language models. GNOT exposes a uniform `POST /action` interface across all nodes: an LLM—whether a cloud-hosted assistant such as Claude, a self-hosted model, or any tool-calling-capable system—can drive the full mesh by issuing structured action requests against this single endpoint, with no framework SDK, no workflow DSL, and no pre-written automation code.

The system bootstraps from a single *seed node* carrying three irreducible primitive capabilities—`read_file`, `write_file`, and `execute_command`—and propagates itself across any reachable machine over HTTP. Complex multi-step workflows are expressed as free-text intents, decomposed autonomously through a ReAct [2] agent loop, and executed across nodes that may span multiple NAT boundaries. Nodes optionally host a configurable LLM provider, activating a `POST /intent` endpoint for embedded orchestration; however, this is an additive capability—the external LLM model is the primary and production-validated interaction mode.

Key architectural contributions include: (1) a *minimal-seed bootstrap model* in which the LLM provisions new compute nodes using only the three primitives; (2) a *BGP-inspired route advertisement protocol* enabling multi-hop NAT traversal without topology prior knowledge; (3) a *transparent push/pull delivery abstraction* that makes network topology invisible to the LLM caller; (4) a *per-session AES-256-GCM credential system* with key derivation scoped to each session; and (5) a *lazy evaluation pattern* applied uniformly to staleness detection, job timeouts, and file expiry. GNOT has been deployed in production across four heterogeneous Linux nodes spanning two NAT layers, with validated use cases including cross-network database migration, Telegram-based CRM integration, autonomous software development, and AI-driven content publishing.

Keywords: distributed systems, large language models, orchestration, autonomous agents, ReAct, NAT traversal, BGP routing, edge computing

ACM CCS: Networks → Network architectures; Computing methodologies → Intelligent agents

1 Introduction

The emergence of instruction-following large language models has created a new class of automation systems variously called “AI agents,” “LLM orchestrators,” and “agentic pipelines.” [1, 2]. The dominant architectural pattern in these systems is a *wrapper model*: the LLM is given a curated set of named tools—API endpoints, function schemas—that it invokes to interact with the external world. Tools are authored by engineers, schemas are static, and the LLM is fundamentally a consumer of a pre-provisioned toolset.

This architecture works well for narrow task domains but exhibits three structural limitations at scale:

- **Static capability boundary.** New capabilities require engineering effort to define and expose as tools. The LLM cannot extend its own toolset at runtime.
- **Topology blindness.** Tools abstract away network topology, making it impossible for the LLM to reason about which physical machine executes which task—relevant for latency, data locality, and resource-constrained deployments.
- **No NAT awareness.** Real enterprise and edge deployments routinely place compute behind NAT layers. Existing frameworks assume reachability and provide no first-class mechanism for NAT traversal.

GNOT addresses these limitations through a different framing: rather than asking *which tools should we give the LLM*, GNOT asks *what interface should the infrastructure expose so that any LLM can orchestrate it natively*. The answer is a uniform `POST /action` endpoint, a self-describing capability tree at `GET /capabilities`, and

a single generic `mesh_action` tool schema—sufficient for any tool-calling-capable LLM to discover and operate the full mesh without SDK integration, workflow configuration, or pre-written automation code.

This design has a practical consequence that distinguishes GNOT from embedded-agent frameworks: *the orchestrating LLM need not run inside the infrastructure at all*. In production, GNOT is primarily operated via Claude through its web interface—a frontier model with a rich capability surface including vision input, voice interaction, sandboxed code execution, and tool use across long contexts. A subscription-based access model is substantially more cost-efficient than per-token API billing for sustained automation workloads. Critically, the mesh interface is stable while the orchestrating model improves continuously, without any changes to node configuration or action code.

Node-local LLM orchestration is an *additive* capability: when a node is configured with an LLM provider (cloud API or local inference server), it activates a `POST /intent` endpoint that runs the ReAct loop internally. This enables embedded agents, offline-capable nodes, and per-node autonomous orchestration—complementing rather than replacing the external model pattern.

The insight driving the bootstrap model is that exactly three primitive capabilities are both necessary and sufficient to provision any computational environment. `read_file` and `write_file` together provide full filesystem access—required to inspect state, write configuration, and install software artifacts. Without either, the LLM cannot observe or modify the machine’s persistent state. `execute_command` provides process execution—required to run installers, start services, invoke package managers, and call external APIs. Without it, no software can be activated regardless of what is written to disk. Removing any one of the three eliminates an irreducible category of operation; together, they form a minimal complete basis: any Linux environment can be bootstrapped from them, and nothing further is needed at seed time. An LLM reasoning over these three primitives becomes both the provisioning layer and the orchestration layer simultaneously, without requiring any infrastructure pre-existing beyond a running GNOT node.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the GNOT architecture. Section 4 describes the routing engine and BGP-inspired route advertisement. Section 5 covers the LLM control plane and ReAct loop. Section 6 describes the action model and plugin contract. Section 7 discusses the security model. Section 8 explains self-bootstrapping. Section 9 describes the file transfer system. Section 10 describes the production deployment and validated use cases. Section 11 discusses design decisions and limitations. Section 12 concludes.

2 Background and Related Work

2.1 LLM Orchestration Frameworks

LangChain [1] is the most widely adopted framework for building LLM-powered applications. Its agent abstraction gives the LLM access to a registered set of tools—Python functions with typed signatures—that it invokes to interact with the external world. Tools are statically defined at application startup; the LLM cannot extend its own toolset at runtime. More fundamentally, LangChain does not address multi-machine execution: all tool calls execute within a single process on a single host. The author evaluated LangChain as a foundation for the use cases described in Section 10 but found it insufficient: there is no mechanism for an LLM to target a specific remote machine, route through NAT, or provision a new node at runtime.

LlamaIndex is primarily a retrieval-augmented generation (RAG) and data indexing framework. Its `llama-deploy` extension supports distributing agents as microservices with a message queue, but requires each service to be pre-defined, containerized, and registered with a central control plane before execution. The framework assumes a pre-provisioned, reachable infrastructure; there is no concept of dynamic node registration, NAT traversal, or LLM-driven self-extension. The author found LlamaIndex insufficient for the same reason as LangChain: it cannot accept an LLM instruction to “install software on machine X and register it as a new execution node.”

OpenClaw [4] (formerly Clawdbot and Moltbot) is a personal AI assistant framework that connects an LLM to messaging platforms (Telegram, WhatsApp, Slack, Discord, and others) and exposes shell, browser, file, and calendar operations as tools. It gained significant adoption in early 2026 and is architecturally relevant to GNOT: both systems route natural-language requests through a local gateway and dispatch them to execution primitives. OpenClaw’s architectural focus, however, is the *single-machine personal assistant*: the Gateway is a WebSocket server on one host, skills execute locally, and there is no concept of remote node registration, cross-machine routing, or NAT-transparent distributed execution. An operator cannot instruct an OpenClaw agent to execute a command on a second private machine and return the result—the class of use cases GNOT is designed for.

ZeroClaw [5] is a Rust-native reimplementation of the OpenClaw architecture, optimized for resource-constrained deployments (3.4 MB binary, sub-10 ms cold start). It shares OpenClaw’s single-machine execution model and messaging-first interface. ZeroClaw’s design priorities—low memory footprint, security-first defaults, sandboxed filesystem access—address operational concerns in single-agent deployments. Like OpenClaw, it does not address multi-machine mesh execution, dynamic node

provisioning, or NAT-transparent routing.

The common limitation across all four frameworks is the *static infrastructure assumption*: the LLM is given tools that call pre-existing, reachable services. GNOT inverts this by making the infrastructure itself the dynamic artifact: the LLM provisions, connects, and orchestrates nodes at runtime, using three primitives that suffice to bootstrap any Linux environment.

2.2 ReAct: Reasoning and Acting

Yao et al. [2] introduced ReAct, a prompting strategy that interleaves reasoning traces (*Thought*) with action invocations (*Act*) and environment observations (*Obs*) in an alternating loop. The pattern allows an LLM to decompose multi-step problems, course-correct on unexpected results, and produce a final answer only when it has sufficient information.

GNOT adopts ReAct as the execution model of its `IntentHandler`. The key extension over the original formulation is the action target: where Yao et al. dispatch to a local function registry, GNOT dispatches to a distributed execution mesh that may span multiple machines across multiple NAT boundaries. The LLM’s reasoning trace therefore includes implicit decisions about *where* to execute, not only *what* to execute.

2.3 BGP and Application-Layer Routing

The Border Gateway Protocol (BGP) [3] solves inter-domain routing on the internet. Its central abstraction is *next-hop forwarding with route advertisement*: each autonomous system announces the IP prefixes it can reach to its neighbors, which propagate the announcement further. A router needs to know only its immediate next hop toward a destination—not the full path.

GNOT maps this abstraction to the application layer. Each intermediate mesh node advertises the sub-nodes reachable through it; the gateway installs these routes with a next-hop pointer. The result is that a request for a deeply nested node traverses the mesh correctly without any node needing global topology knowledge. Section 4.3 describes the protocol in detail.

3 System Architecture

3.1 Design Principles

GNOT is founded on five principles:

1. **Maximal minimalism at origin.** The seed node carries exactly three capabilities. Everything else emerges at runtime.
2. **LLM-native interface.** The mesh exposes a uniform, self-describing interface (`POST /action`, `GET`

`/capabilities`, `mesh.action` tool schema) designed so that any tool-calling-capable LLM can orchestrate it natively—without SDK wrappers, static tool registration, or workflow configuration code.

3. **Transparent delivery.** Whether execution is local, synchronous remote (push), or asynchronous queued (pull), the calling interface is identical: `POST /action` returning either an inline result or a `job_id` for polling.
4. **Composition over configuration.** No workflow DSL, no DAG executor. The LLM composes multi-step workflows through sequential tool calls.
5. **LLM flexibility.** The mesh does not mandate where the orchestrating LLM runs. An external LLM—cloud-hosted, local, or subscription-accessed—is the primary and production-validated mode. A node-local LLM (activated via `llm_provider` configuration) is an additive option for embedded or offline-capable deployments. Both interact with the mesh through the same routing layer.

3.2 System Architecture Overview

Figure 1 presents the complete GNOT architecture. A node is composed of three concentric layers: the *API layer* that receives all inbound requests, the *runtime layer* that implements routing, job management, session state, and credential storage, and the *execution layer* that runs action code. Nodes connect to each other exclusively through their API layers; internal components are never exposed directly.

3.3 Node Anatomy

Every GNOT node is an HTTP service running on a Linux machine. Its filesystem structure is minimal:

```
/opt/mesh/<node-id>/
node.yaml          # configuration
actions/
  execute_command.py
  execute_command.schema.json
  read_file.py
  read_file.schema.json
  write_file.py
  write_file.schema.json
  <custom_action>.py      # optional
  <custom_action>.schema.json # optional
```

Listing 1: Node directory structure

The runtime (`node_runtime.py` and the `runtime/` package) is shared across all nodes on a machine and installed once. Nodes are differentiated only by their `node.yaml` configuration and their `actions/` directory contents.

3.4 Node Operating Modes

A node operates in one or more modes simultaneously, determined solely by its configuration (Table 1).

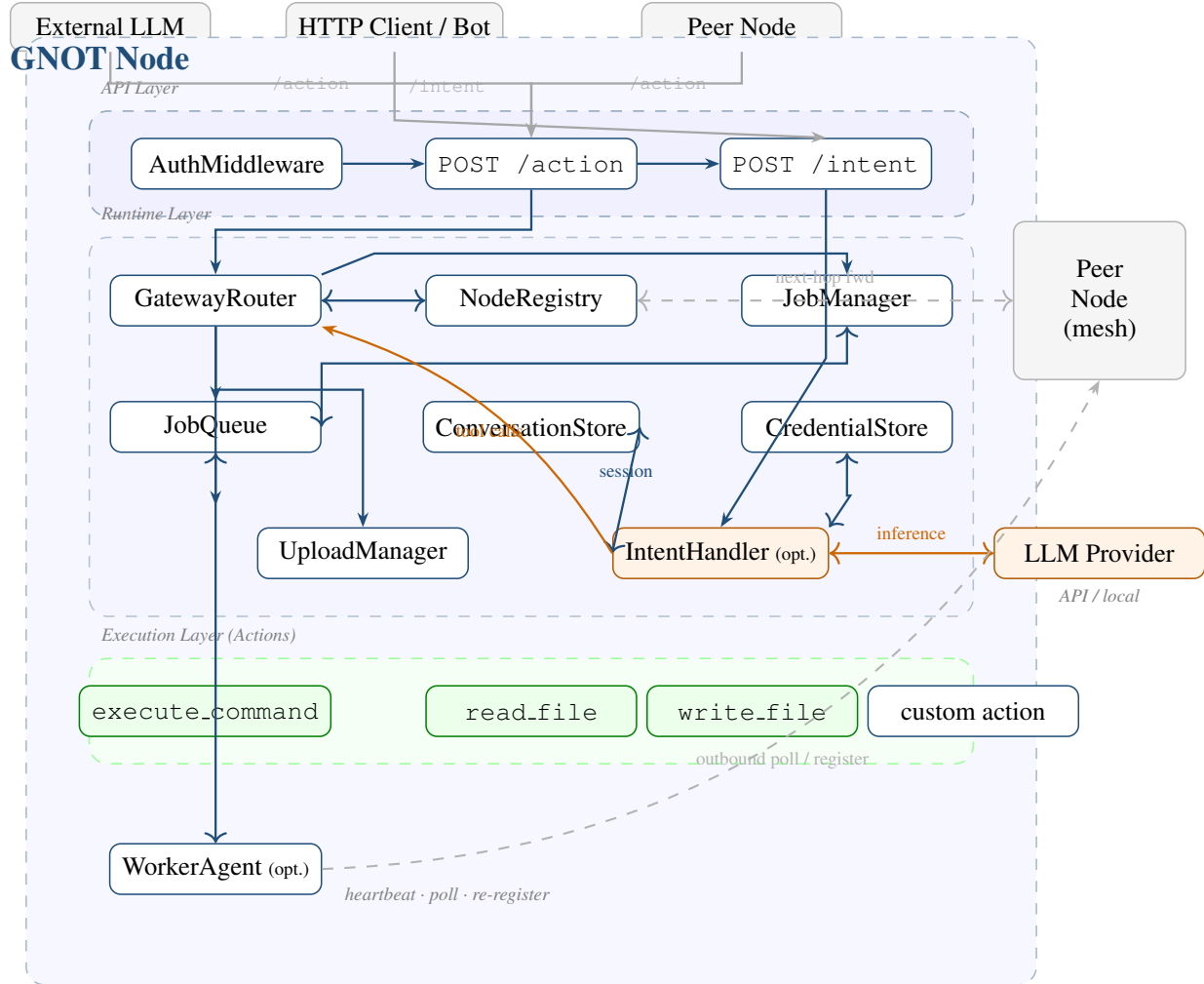


Figure 1: GNOT system architecture overview. Three layers handle all request processing: the API layer (AuthMiddleware, endpoints), the Runtime layer (routing, job management, session and credential stores), and the Execution layer (seed actions and custom plugins). The IntentHandler and LLM Provider (orange) are optional, activated only when `llm_provider` is configured. The WorkerAgent (optional) enables pull-mode operation for nodes behind NAT. Dashed arrows indicate optional or asynchronous flows.

Table 1: Node operating modes

Mode	Condition and Role
Standalone	No peers configured. Executes actions locally only.
Gateway	<code>trusted_nodes</code> configured. Accepts worker registrations, routes requests, hosts IntentHandler.
Worker	<code>gateway_node_id</code> + <code>gateway_address</code> set. Polls gateway, executes jobs.
Gateway+Worker	Both conditions. Intermediate node: routes to sub-nodes and is itself a worker.

3.5 Topology Overview

Figure 3 illustrates a representative GNOT deployment. A gateway node is publicly reachable (via Cloudflare Tunnel or direct IP) and accepts requests from external callers—web interfaces, Telegram bots, or HTTP clients. Worker nodes behind one or more NAT layers connect outward to the gateway; they never receive inbound connections.

3.6 LLM Interaction Model

GNOT is designed so that the LLM orchestrator is *decoupled from the node runtime*. Every node exposes `POST /action` unconditionally; `POST /intent` is an optional endpoint, activated only when an `llm_provider` is configured. This yields two complementary interaction

patterns (Figure 2):

Pattern A — External LLM (primary). An LLM operating outside the mesh acts as the orchestrator. The user provides the LLM with the mesh capability description from `GET /capabilities` and the `mesh_action` tool schema. The LLM then drives the mesh directly via `POST /action`, carrying `target_node_id`, action name, and parameters per request. The node runtime executes, routes, or queues each request with no LLM involvement on any node.

This is the primary and production-validated pattern. In the reference deployment, Claude—accessed via its web interface—serves as the orchestrator. This approach offers several practical advantages over embedding an LLM inside the infrastructure:

- **Richer capability surface.** A frontier model accessed through a full-featured interface provides vision input, voice interaction, sandboxed code execution, extended context windows, and continuous model improvements—all without changes to node configuration.
- **Cost efficiency.** Subscription-based access to a frontier model is substantially more economical than per-token API billing for sustained multi-step automation workloads.
- **Zero inference overhead on mesh nodes.** Execution nodes remain lightweight and resource-predictable; no GPU or model serving infrastructure is required anywhere in the mesh.

Pattern B — Node-local LLM (additive). When a node is configured with an `llm_provider` (external API or local inference server such as Ollama or vLLM), it activates the `IntentHandler` and exposes `POST /intent`. A caller submits a free-text prompt; the node’s LLM decomposes it into `mesh_action` tool calls, dispatches them through the same `GatewayRouter` as Pattern A, and returns a synthesized natural-language response. This pattern suits use cases requiring low-latency embedded agents, air-gapped or offline deployments, or per-node autonomous decision making.

The two patterns are not mutually exclusive: some nodes may run node-local LLMs for embedded processing while others operate as pure execution nodes reachable from any external orchestrator. The routing layer is identical in both cases.

3.7 Principal Components

Table 2 summarizes the principal runtime components.

3.8 Capability Discovery

Every GNOT node exposes a `GET /capabilities` endpoint that returns the complete capability tree of the

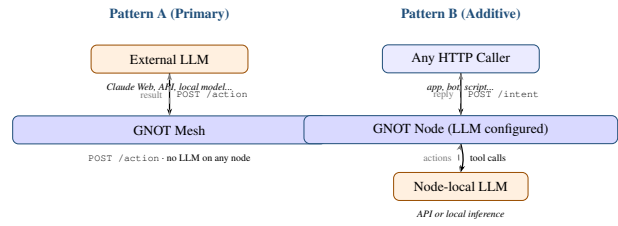


Figure 2: LLM interaction patterns. Pattern A (primary): an external LLM orchestrates the full mesh via `POST /action`; no LLM is required on any node. Pattern B (additive): a configured node-local LLM processes free-text intents via `POST /intent`, dispatching the same `mesh_action` tool calls through the same routing layer. Both patterns can coexist in the same deployment.

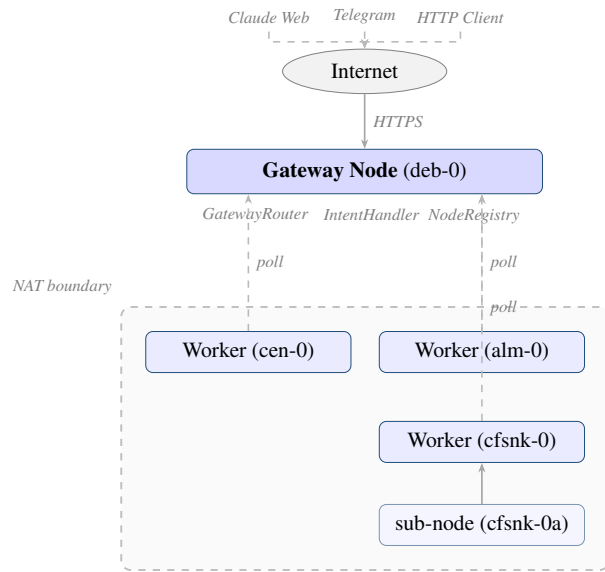


Figure 3: GNOT deployment topology. Workers poll the gateway outbound (dashed arrows); the gateway never initiates inbound connections to workers behind NAT. Sub-node `cfsnk-0a` is reachable only via its parent `cfsnk-0`.

Table 2: Principal GNOT components

Component	Responsibility
GatewayRouter	Routes POST /action to correct delivery path: local / push / pull / next-hop forward.
NodeRegistry	Tracks trusted nodes, liveness, addresses, capabilities, and BGP next-hop entries.
JobManager	Maintains job state machine (accepted → running → completed/failed).
JobQueue	Per-node pull queue; holds jobs for workers that cannot receive inbound connections.
WorkerAgent	Three async tasks inside each worker: registration, heartbeat, poll-execute-report.
IntentHandler	ReAct loop: converts free-text prompts into mesh action sequences.
ConversationStore	Per-session message history for multi-turn conversations.
CredentialStore	AES-256-GCM encrypted store for caller-supplied credentials.
UploadManager	Staged binary file store enabling large-file cross-NAT transfer.

node and all nodes reachable through it—including action specifications, parameter schemas, caller credential requirements, liveness status, and next-hop routing information:

```
GET /capabilities
{
  "node_id": "deb-0",
  "actions": ["execute_command", "read_file", "write_file"],
  "reachable": {
    "cen-0": {
      "status": "online",
      "next_hop": null,
      "actions": ["execute_command", "read_file", "write_file"],
      "action_specs": {
        "execute_command": {
          "description": "Execute shell command",
          "params_schema": {
            "command": { "type": "string" },
            "timeout_seconds": { "type": "integer", "default": 60 }
          },
          "async_action": true
        }
      }
    },
    "cfsnk-0a": {
      "status": "online",
      "next_hop": "cfsnk-0",
      "actions": ["get_order_info"],
      "action_specs": {
        "get_order_info": {
          "caller_credentials": {
            "crm_user_token": {
              "description": "CRM API key",
              "required": true
            }
          }
        }
      }
    }
  }
}
```

```
}
}
}
}
```

Listing 2: Capability tree response (excerpt)

The status field reflects a lazy staleness check at query time (Section 4.5). The next_hop field exposes the BGP-style routing entry transparently: a caller targeting cfsnk-0a need not know that cfsnk-0 is the intermediate node—the gateway handles forwarding automatically.

GET /capabilities is the primary mechanism by which an external LLM (Model A, Section 3.6) learns the mesh topology before issuing POST /action calls. For Model B, the IntentHandler calls this endpoint internally to generate the dynamic system prompt at each POST /intent request.

A legacy GET /skills endpoint returns a human-readable Markdown description of node capabilities, retained for backward compatibility with external LLM workflows that inject skills as freeform context rather than structured JSON.

3.9 Observability

GET /health returns a structured summary of node state:

```
{
  "node_id": "deb-0",
  "status": "healthy",
  "uptime_seconds": 3600.5,
  "actions_loaded": 5,
  "jobs_active": 2,
  "queue_depths": {
    "cen-0": 3,
    "alm-0": 0,
    "cfsnk-0": 1
  }
}
```

Listing 3: Health endpoint response

The queue_depths field reports unclaimed jobs per worker node in the pull queue. This field serves three operational purposes: detecting stuck workers (depth growing while worker is offline), triggering scale-out decisions (depth exceeding threshold → bootstrap new worker via execute_command), and general health monitoring. At the L3 evolution level (Section 8), an LLM observing queue_depths can autonomously decide to provision additional worker nodes and decommission them when load subsides—closing the self-optimization loop entirely within the mesh.

4 Routing Engine

4.1 Request Envelope

Every action request follows a uniform envelope:

```
{
  "target_node_id": "cfsnk-0a",
  "payload": {
    "action": "execute_command",
    "params": { "command": "df -h" }
  },
  "task_id": "task-3f8a91c2b4e1", // auto-generated if absent
  "trace": { "hop_count": 0, "route_path": [] },
  "caller_credentials": {}
}
```

Listing 4: Action request envelope

The response is either synchronous (inline output) or asynchronous (a `job_id` for polling). The *presence of* `job_id` is the sole indicator of async mode; the caller sets no mode flag.

4.2 GatewayRouter Decision Tree

Every node runs a `GatewayRouter` that applies the following decision tree on every `POST /action`:

```
route(request):
1. if hop_count > max_hop:      MAX_HOP_EXCEEDED
2. if node_id in route_path:    LOOP_DETECTED
3. if target == self:          execute_local()
4. next_hop = registry.get_next_hop(target)
   if next_hop != target:
       forward_via_next_hop()
5. if not registry.is_trusted(target):
   UNTRUSTED_NODE
6. address = registry.get_address(target)
   if not address:              pull(request)
   elif ping(target):           result = push(
       address, request)
   if error:                    pull(request)
   // fallback
   else:                         pull(request)
```

Listing 5: Routing decision logic (pseudocode)

Steps 1–2 prevent routing loops. Step 4 implements the BGP-inspired next-hop forwarding described in Section 4.3. Step 6 provides the transparent push/pull abstraction described in Section 4.4.

4.3 BGP-Inspired Route Advertisement

A fundamental challenge in heterogeneous network deployments is that the gateway may not have direct connectivity to all nodes. Consider a three-layer topology:

gateway (deb-0) → node-1 → node-1a

Without any route advertisement mechanism, the gateway cannot route requests to node-1a: it has no entry for that node. GNOT solves this with a protocol analogous to BGP route advertisement.

Table 3: BGP-to-GNOT concept mapping

BGP Concept	GNOT Equivalent
Autonomous System	Node
IP prefix	Node ID
BGP neighbor	Gateway node
Route advertisement	advertise_routes in <code>POST /nodes/register</code>
Next-hop router	next_hop in <code>NodeRegistry</code> entry
BGP UPDATE message	Periodic <code>WorkerAgent</code> re-registration
BGP WITHDRAW message	Re-registration with reduced <code>advertise_routes</code>

Advertisement flow. When node-1a registers with node-1, it includes its action list. node-1’s `WorkerAgent` then re-registers with the gateway, including node-1a in its `advertise_routes` list:

```
POST gateway/nodes/register {
  "node_id": "node-1",
  "actions": ["execute_command"],
  "advertise_routes": ["node-1a"],
  "capabilities": {
    "node-1a": ["execute_command", "get_order_info"]
  },
  "sub_route_specs": {
    "node-1a": { "get_order_info": { ...full spec ... } }
  }
}
```

Listing 6: Re-registration with route advertisement

The gateway installs node-1a with `next_hop = node-1`. Subsequent requests for node-1a are forwarded to node-1, which then routes to node-1a directly.

BGP analogy. Table 3 maps BGP concepts to their GNOT equivalents.

Route withdrawal. When a sub-node goes offline, its parent detects the absence via heartbeat timeout and excludes it from the next re-registration. The gateway compares the new advertisement against existing entries and withdraws the difference:

```
def withdraw_routes(via_node, keep_routes):
    withdrawn = [
        nid for nid, entry in _entries.items()
        if entry.next_hop == via_node
        and nid not in keep_routes
    ]
    for nid in withdrawn:
        del _entries[nid]
    return withdrawn
```

Listing 7: Route withdrawal logic

Action spec cascade. Full action specifications—descriptions, parameter schemas, and caller credential requirements—propagate through multi-hop re-advertisement via the `sub_route_specs` field, ensuring the gateway’s capability tree reflects the complete mesh capability even for deeply nested nodes.

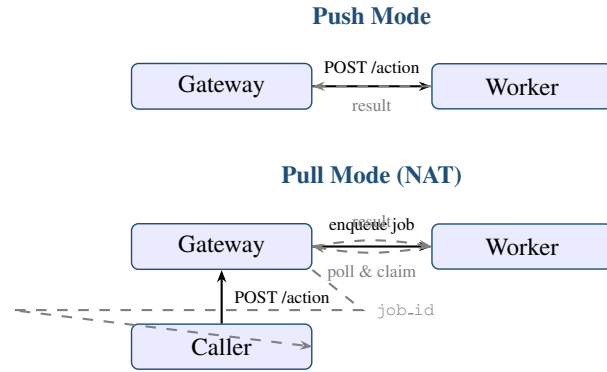


Figure 4: Push mode (direct proxy) vs. pull mode (queue-based) delivery.

4.4 Push/Pull Delivery Model

NAT traversal is the fundamental connectivity challenge in heterogeneous deployments. GNOT addresses it with a two-mode delivery model:

Push mode (Figure 4, top): When a worker has a known, reachable address, the gateway proxies the action directly. The gateway pings the worker before committing to push; on failure it falls back to pull transparently.

Pull mode (Figure 4, bottom): When a worker is behind NAT or unreachable, the gateway enqueues the job and returns a `job.id` immediately. The worker's `WorkerAgent` polls `GET /jobs/poll` every 5 seconds, claims available jobs atomically, executes locally, and reports results.

Claim atomicity. Multiple worker replicas polling concurrently are safe: job claiming is protected by an `asyncio lock`, and only the first `POST /jobs/{job.id}/claim` succeeds.

Pull job timeout. Jobs that remain unclaimed beyond `pull_job_timeout_seconds` are marked failed lazily at `GET /result` query time—consistent with the system-wide lazy evaluation principle described in Section 4.5.

4.5 Lazy Evaluation Pattern

A recurring design choice in GNOT is *lazy evaluation*: rather than maintaining background processes to sweep for stale state, GNOT applies state checks at the moment the state is consulted. This pattern is applied uniformly to:

- **Node staleness:** A node's heartbeat age is checked at routing time, not by a background timer.
- **Pull job timeout:** Checked at `GET /result` time, not by a background cleanup loop.
- **Upload TTL:** Expired files are removed on listing and on `GET /download`, not by a background sweeper.

- **Session expiry:** Sessions are silently recreated at `get_or_create()` time if expired.

Lazy evaluation eliminates background task coordination overhead, makes behavior deterministic and testable, and ensures that routing decisions are made with information current at the moment they matter.

5 LLM Control Plane

5.1 LLM Provider Configuration

`POST /intent` is available only on nodes that have an LLM provider configured in `node.yaml`. GNOT supports any OpenAI-compatible API as a provider, covering a wide range of deployment scenarios:

```

# External API (Anthropic Claude)
llm_provider: anthropic
llm_model: claude-sonnet-4-5
llm_api_key: sk-ant-...

# External API (OpenAI)
llm_provider: openai
llm_model: gpt-4o
llm_api_key: sk-...

# Local inference (Ollama)
llm_provider: ollama
llm_model: llama3.1:8b
llm_base_url: http://localhost:11434/v1

# Local inference (vLLM, LM Studio, etc.)
llm_provider: openai_compatible
llm_model: mistral-7b-instruct
llm_base_url: http://localhost:8000/v1

```

Listing 8: LLM provider configuration examples

A node with no `llm_provider` configured is a pure execution node: it exposes `POST /action` only and incurs zero inference cost. This makes GNOT deployable on resource-constrained edge machines that cannot run or afford LLM inference, while still participating fully in the mesh as action executors (see Section 3.6).

5.2 Intent Handling and ReAct Loop

`POST /intent` is the primary human-facing endpoint. It accepts a free-text prompt and executes a ReAct loop internally:

```

handle(request):
    session = get_or_create(session_id)
    session.add("user", prompt)
    system_prompt = build_from_live_capability_tree()

    for turn in range(max_turns):
        response = llm.chat(session.messages,
                             tools=[MESH_TOOL],
                             temperature=0.2)

        if response.tool_calls:
            for call in response.tool_calls:
                result = router.route(ActionRequest(call))
                if async: poll until complete
                session.add_tool_result(result)
                continue // next turn
            else:

```

```

return IntentResponse(reply=response.text)

return IntentResponse(truncated=True)

```

Listing 9: ReAct loop pseudocode

The loop terminates when the LLM produces a plain-text response (no tool calls) or when `max_turns` is reached.

5.3 Single Generic Tool Design

A key design decision is exposing a single `mesh_action` tool to the LLM rather than one tool per action per node:

```

{
  "name": "mesh_action",
  "parameters": {
    "target_node_id": "string",
    "action": "string",
    "params": "object"
  }
}

```

Listing 10: The `mesh_action` tool schema

This design has two important properties. First, the tool registration is *stable*: new nodes and new actions join the mesh without modifying the tool schema. Second, the LLM acquires capability knowledge from the dynamically generated system prompt rather than from static tool definitions, keeping the tool interface minimal.

5.4 Dynamic System Prompt

The `IntentHandler` builds the system prompt from the live capability tree at each request, including node liveness status:

```

## Mesh topology
- deb-0 [gateway, THIS NODE]
  - execute_command: Execute shell command
  - read_file, write_file

- cen-0 (direct, online)
  - execute_command, read_file, write_file

- cfsnk-0a via cfsnk-0 (online)
  - get_order_info: Retrieve CRM order details
    param order_id (string, required)
    [CREDENTIAL REQUIRED] crm_user_token

- cfsnk-0b via cfsnk-0 [UNREACHABLE]
  - send_notification: ...

```

Listing 11: Excerpt of generated system prompt

Nodes marked `[UNREACHABLE]` are excluded from routing; the LLM does not attempt to call them, avoiding multi-minute timeout waits.

5.5 Session Management

Sessions are identified by a caller-supplied `session_id` (e.g., a Telegram `chat_id` or `UUID`). The `ConversationStore` maintains the full OpenAI-format message history across multiple `POST /intent`

calls, enabling stateful multi-turn conversations. Sessions expire after a configurable TTL; expiry is checked lazily at session access time.

6 Action Model

6.1 Seed Actions

Three primitive actions constitute the bootstrap kernel and are present on every node:

- `execute_command`: Executes an arbitrary shell command and returns `stdout`, `stderr`, and `exit_code`. This is the most powerful primitive: it enables installing software, writing configurations, starting processes, and bootstrapping new GNOT nodes.
- `read_file`: Reads and returns the text content of a file at a specified path.
- `write_file`: Writes text content to a file, creating parent directories as needed.

6.2 Plugin Action Contract

Custom actions are Python modules placed in a node's `actions/` directory. The runtime loads them at startup via the following contract:

```

# Synchronous action
def run(params: dict, context: dict) -> dict: ...

# Asynchronous action
ASYNC = True
async def run(params: dict, context: dict) -> dict:
    ...

```

Listing 12: Plugin action contract

The context dict is injected by the runtime and includes: `task_id`, `job_id`, `node_id`, the full `NodeConfig`, a shared `LLMClient`, a temporary directory path, and `caller_credentials`.

6.3 JSON Schema Validation and Discovery

Each action may have a co-located `{action}.schema.json` file (JSON Schema Draft 7). Schemas serve three purposes: (1) request validation before execution, returning HTTP 422 on failure; (2) LLM guidance via the `GET /capabilities` endpoint and system prompt; (3) caller credential declaration via the `x-caller-credentials` vendor extension.

7 Security Model

7.1 Authentication Layers

GNOT implements security at two layers:

HTTP layer (AuthMiddleware). Every non-exempt endpoint requires a Bearer token. GNOT v5.13 introduced a per-node token model: the gateway maintains an `allowed_tokens` list, and each worker has a unique token. This enables single-node revocation without cluster-wide rotation. Token comparison uses `secrets.compare_digest` (constant-time) to prevent timing attacks.

Action layer (CallerPolicies). Individual actions can be further restricted per-caller-token via `caller_policies` in `node.yaml`:

```
caller_policies:
- token: "sk-sales-team"
  allowed_actions: [get_order_info, list_orders]
- token: "sk-devops"
  allowed_actions: [execute_command, read_file]
- token: "sk-admin"
  allowed_actions: "*"

```

Listing 13: Action-level authorization example

7.2 Caller Credential System

Some actions require credentials that the *caller* supplies—for example, a CRM API token that the calling user must own. These are distinct from service credentials (database passwords, etc.) managed by the action internally.

Caller credentials are transported in the `caller_credentials` field of `ActionRequest`, propagated through the full push/pull chain, injected into action context, and *never logged, never stored in params, never appear in response bodies*.

7.3 AES-256-GCM Credential Storage

For multi-turn sessions via `POST /intent`, the `CredentialStore` allows a caller to supply credentials once and reuse them across subsequent turns. Credentials are stored encrypted using AES-256-GCM with per-session key derivation:

$$k_{\text{session}} = \text{SHA-256}(\text{enc_key} \parallel : \parallel \text{session_id}) \quad (1)$$

Per-session key derivation (Eq. 1) ensures that compromising one session’s key does not affect other sessions. The encryption key (`credential_encryption_key`) is separate from the authentication token (`auth_token`), allowing each to be rotated independently.

Persistence uses a write-through pattern with atomic file rename: credentials are written to a `.tmp` file and atomically renamed to the target path, ensuring readers never observe a partial write.

Table 4: Cluster evolution levels

Level	State	Description
L0	Seed Only	Single node, 3 actions
L1	Functional Nodes	Domain-specific nodes (DB, API, ML)
L2	Specialized Clusters	GPU, storage, CPU-heavy workers
L3	Self-Optimization	LLM replaces slow nodes, scales on queue depth

8 Self-Bootstrapping

8.1 Seed-to-Mesh Bootstrap

The three seed actions are sufficient to create any new GNOT node. The LLM orchestrates the following sequence via sequential `mesh_action` tool calls:

```
1. execute_command: "mkdir -p /opt/mesh/node-B/actions"
2. write_file: path=/opt/mesh/node-B/node.yaml
   content=<generated node config>
3. write_file: path=../actions/custom.py
   content=<generated action code>
4. execute_command: "pip install <deps>"
5. execute_command: "nohup python node_runtime.py \
   --config node-B/node.yaml &"
6. execute_command: "curl -sf localhost:8081/health"

```

Listing 14: LLM-orchestrated node bootstrap sequence

No external provisioning tool, configuration management system, or container orchestrator is required. The LLM is the provisioning layer.

8.2 One-Liner Worker Enrollment

Any Linux machine can join the mesh with a single command:

```
curl -sSL https://gateway.example.com/setup.sh \
| bash -s -- --node-id cen-0 \
--auth-token "$TOKEN" \
--systemd

```

`GET /setup.sh` returns a universal setup script (auto-injected with the gateway URL). The script handles OS detection (Debian/Ubuntu, CentOS/RHEL, AlmaLinux, Arch, Alpine), Python 3.10+ installation, virtualenv creation, runtime download, `node.yaml` generation, and `systemd` service creation. Both `/setup.sh` and `/runtime-bundle` are authentication-exempt so they can be fetched before a token is available.

8.3 Evolution Levels

GNOT clusters evolve through four levels:

At L3, the LLM can autonomously decide to bootstrap additional worker nodes when it observes growing queue

depth in GET /health's queue_depths field, then decommission them when load subsides.

9 File Transfer System

The seed action `write_file` writes content inline in a JSON payload, practical for text files up to approximately 1 MB. For binary files (database dumps, compressed archives) at hundreds of megabytes, GNOT provides a dedicated staged transfer mechanism.

The gateway's `UploadManager` accepts multipart/form-data uploads and returns a `file_id`. Workers on separate NAT layers can then pull the file via GET /download/{file_id}:

```
# Step 1: Worker cen-0 uploads backup to gateway
curl -X POST https://gw/upload \
-H "Authorization: Bearer $TOKEN" \
-F "file=@/tmp/backup.sql.gz"
# => { "file_id": "a3f9b2c1d4e5f678" }

# Step 2: Worker alm-0 downloads from gateway
curl -OJ https://gw/download/a3f9b2c1d4e5f678 \
-H "Authorization: Bearer $TOKEN"
```

Listing 15: Cross-NAT binary file transfer

This mechanism enables complete workflows such as cross-NAT database migration orchestrated by a single POST /intent.

10 Production Deployment

GNOT has been deployed in a production environment. The reference cluster comprises four heterogeneous Linux nodes:

All three workers operate behind NAT and are therefore pull-only: they never receive inbound connections. The gateway is exposed via a Cloudflare Tunnel, requiring no port forwarding or public IP on the host machine. Workers make outbound HTTPS connections to the gateway exclusively.

10.1 Production Use Cases

The following four use cases have been validated on the production cluster. Each is initiated by a single natural-language prompt with zero pre-written workflow code. They are presented in order of increasing architectural complexity.

10.1.1 Cross-Network Database Migration

Two worker nodes (cen-0 and cfsnk-0) reside on separate private networks with no direct connectivity. A MySQL database hosted on cfsnk-0 must be migrated to cen-0.

The LLM orchestrates the following sequence entirely through the mesh:

Table 5: Reference production cluster

Node	OS	Mode	Connectivity
deb-0	Debian	Gateway	Public via Cloudflare Tunnel
cen-0	CentOS	Worker	Private, pull-only
alm-0	AlmaLinux	Worker	Private, pull-only
cfsnk-0	CentOS	Worker	Private, pull-only

1. `execute_command` on cfsnk-0: `mysqldump > /tmp/db.sql.gz` — exports and compresses the database.
2. POST /upload to gateway (deb-0): transfers the dump file via the staged binary upload mechanism.
3. GET /download/{file_id} from cen-0: worker pulls the dump from the gateway.
4. `execute_command` on cen-0: restores the dump into the target MySQL instance.

This workflow crosses two NAT boundaries (cfsnk-0 → gateway → cen-0) without any direct tunnel or VPN between the two machines. The `UploadManager`'s staged transfer (Section 9) acts as the data relay point.

10.1.2 Telegram-Based Order Tracking

A customer-facing Telegram bot is integrated with an internal CRM system running on a private network node (cfsnk-0a). The gateway node (deb-0) runs both the Telegram webhook handler and the `IntentHandler`.

When a customer sends an order number via Telegram, the bot forwards the message as a POST /intent request to the gateway. The LLM resolves the intent, calls `get_order_info` on cfsnk-0a via the pull-mode delivery chain, and returns the result as a natural-language Telegram reply. The CRM node never requires a public address; the pull-mode delivery model (Section 4.4) handles connectivity transparently.

The customer's CRM API token is supplied once per Telegram session via `caller_credentials` and stored encrypted in the `CredentialStore` for the session duration, so subsequent queries within the same session require no re-authentication.

10.1.3 Autonomous Software Development

A developer provides a natural-language feature description or bug report via POST /intent. The LLM autonomously performs the complete development cycle using only the three seed actions:

1. `read_file` — inspects relevant source files and test suites in the target codebase.
2. `write_file` — writes new source files or applies targeted fixes to existing ones.

3. `execute_command` — runs the test suite (`python -m pytest`) and re-iterates if tests fail.
4. `execute_command` — deploys the change (restart service, run migration, or copy artifacts) on the same node or a designated deployment node.

The LLM’s ReAct loop handles test failures as observations: if `pytest` returns a non-zero exit code, the LLM reads the failure output, revises the implementation, and re-runs tests—all within the same `POST /intent` turn sequence, without any human intervention. Both greenfield feature development and bug fixing have been validated with this approach.

10.1.4 Automated YouTube Content Pipeline

A content creator provides a topic as a single natural-language prompt. The LLM orchestrates a complete content production and publishing pipeline across mesh nodes:

1. **Research:** `execute_command` invokes a web search or summarization script to gather topic material.
2. **Script generation:** The LLM generates a narration script, which `write_file` saves to the working node.
3. **Media generation:** `execute_command` calls an AI image/video generation API (e.g., Stable Diffusion, text-to-video model) to produce visual assets for each segment.
4. **Video rendering:** `execute_command` invokes `ffmpeg` to assemble narration audio (TTS) and visual assets into a final video file.
5. **Upload:** `execute_command` calls the YouTube Data API v3 to upload the rendered video with generated title, description, and tags.

This use case demonstrates that GNOT’s orchestration model extends naturally to creative and media workflows. The entire pipeline—from topic input to published video—runs without the creator writing or executing any code. The mesh nodes provide the compute; the LLM provides the workflow logic.

10.1.5 Summary

Table 6 summarizes the four use cases by the architectural features they exercise.

No use case required any pre-written workflow code, DAG definition, or configuration beyond the standard `node.yaml`. All orchestration logic was generated at run-time by the LLM from a single natural-language prompt.

Table 6: Production use cases and architectural features exercised

Feature	DB Migrate	Telegram CRM	Auto Coding	YouTube
Cross-NAT delivery	✓	✓		
Pull-mode workers	✓	✓	✓	✓
Binary file transfer	✓			✓
Caller credentials		✓		
Multi-turn ReAct loop			✓	✓
External API calls				✓
Session persistence		✓		
Seed actions only	✓		✓	✓

11 Discussion

11.1 Design Decisions

D1: Why lazy evaluation over background polling?

Background staleness detection requires coordination between the detection task and the routing path. Lazy evaluation removes this coordination entirely: state is always consistent at the moment it is read, and the system has one fewer category of concurrency bug.

D2: Why the BGP analogy? BGP solves an analogous problem at internet scale: enabling arbitrary routers to reach arbitrary destinations without knowing full paths. The key insight—that routers need only know the *next hop*, not the full path—maps cleanly to GNOT’s requirement that the gateway need only know which intermediate node to forward to, not the full path to the target.

D3: Why a single mesh action tool? Per-action tools would require the tool registration to be updated whenever the mesh topology changes. A single generic tool with a stable schema decouples the tool interface from the capability surface, allowing the mesh to grow dynamically without LLM re-prompting overhead.

D4: Why AES-256-GCM over a KMS? KMS integration introduces deployment dependencies (IAM roles, network policies, service availability) incompatible with GNOT’s design goal of minimal operational footprint. AES-256-GCM with per-session key derivation provides strong isolation at the cost of a single symmetric secret.

D5: Why separate credential encryption key from auth token? Security best practice requires rotating authentication tokens periodically. Without separation, token rotation would invalidate all active sessions—a non-obvious side effect. Separating the two keys allows each to be rotated on its own schedule.

D6: Why per-node tokens rather than a shared clus-

ter token? A shared cluster token means revoking a compromised worker requires a cluster-wide rotation, temporarily disrupting all nodes. Per-node tokens allow the gateway to revoke a single worker by removing its token from `allowed.tokens`, leaving all other nodes unaffected. The implementation overhead is minimal: token validation is a `frozenset` membership lookup per request.

11.2 Limitations

The current implementation has three known limitations of architectural significance:

- **No container isolation.** `execute_command` has full host filesystem access. The intended mitigation is to run each node as a dedicated non-root OS user with restricted permissions. Container-level isolation via Docker or nsjail would require careful integration to avoid breaking the seed bootstrapping sequence, which itself uses `execute_command` to install software and start processes.
- **In-memory pull queue.** The `JobQueue` is held in process memory. A gateway restart loses all undelivered queued jobs. The configurable `pull_job_timeout_seconds` bounds the exposure window for any individual job, but jobs enqueued immediately before a restart are silently dropped. A SQLite-backed durable queue is planned.
- **Hot-reload configuration.** Adding or revoking tokens in `node.yaml` requires a node restart to take effect. `AuthMiddleware` loads the `allowed.tokens` `frozenset` once at startup. A `SIGHUP` handler or `inotify watch` on `node.yaml` would resolve this without downtime.

11.3 Future Work

Table 7 lists planned improvements by priority.

The multi-network membership model (one node joining multiple gateways), event emit/subscribe between nodes, and a built-in scheduler for periodic action invocation are planned as the primary architectural extensions in GNOT v2.

Evolution context. GNOT reached its current architecture through 14 incremental versions (v5.0–v5.13b). Each version added one well-scoped capability: v5.3 introduced push/pull delivery, v5.7 added lazy staleness detection, v5.9 added the `POST /intent` ReAct loop, v5.10 added BGP-style route advertisement, v5.12 added AES-256-GCM credential encryption, and v5.13b introduced the per-node token model. This incremental approach ensured each mechanism could be validated independently before the next layer was added.

Table 7: Planned improvements

Pri.	Feature	Description
P1	SSE streaming	<code>POST /intent</code> returns <code>text/event-stream</code> with turn-by-turn updates
P1	Durable job queue	SQLite-backed <code>JobQueue</code> surviving gateway restarts
P2	Hot-reload config	<code>SIGHUP</code> or <code>inotify watch</code> on <code>node.yaml</code> for token changes
P2	Token audit logging	Map token fingerprint $\rightarrow$ node identity in access logs
P3	Container isolation	Optionally run actions in Docker or nsjail per node
P3	Metrics endpoint	Prometheus <code>/metrics</code> : job throughput, queue depths, latency
P3	Topology dashboard	Read-only web UI for mesh topology and job history

12 Conclusion

We have presented GNOT, a distributed execution mesh architected for native LLM orchestration. Rather than wrapping infrastructure in LLM-specific tool SDKs, GNOT exposes a uniform, self-describing interface—`POST /action`, `GET /capabilities`, and a single `mesh.action` tool schema—that any tool-calling-capable LLM can operate without framework integration or pre-written workflow code.

The key architectural contributions—minimal-seed bootstrap, BGP-inspired NAT traversal, transparent push/pull delivery, lazy evaluation, and per-session AES-256-GCM credentials—combine into a coherent system that is simultaneously simple to deploy (a single Python process per node) and capable of coordinating complex workflows across machines that have never been directly connected.

Production experience with the reference four-node cluster demonstrates that using a frontier model—Claude, accessed via its web interface—as the external orchestrator yields strong practical results: the full capability surface of a continuously improving model is available at subscription cost, with zero inference overhead on any mesh node. The four validated use cases (cross-network database migration, Telegram CRM integration, autonomous software development, and automated content publishing) were each accomplished through a single natural-language prompt with no pre-written workflow code.

We release the reference implementation at <https://github.com/gnot-io/gnot> [6] under the Apache 2.0 license and invite the research community to explore the broader space of LLM-native distributed infrastructure.

Acknowledgments

The author thanks the open-source communities behind FastAPI, Pydantic, httpx, and the broader Python async ecosystem, without which this implementation would not have been feasible at the present scale.

References

- [1] Chase, H. (2022). *LangChain: Building applications with LLMs through composability*. <https://github.com/langchain-ai/langchain>
- [2] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. <https://arxiv.org/abs/2210.03629>
- [3] Rekhter, Y. and Li, T. (1995). A Border Gateway Protocol 4 (BGP-4). *RFC 1771*, Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc1771>
- [4] Steinberger, P. et al. (2026). *OpenClaw: Open-source autonomous AI agent framework*. openclaw/openclaw, GitHub. MIT License. <https://github.com/openclaw/openclaw>
- [5] ZeroClaw Labs (2026). *ZeroClaw: Fast, small, and fully autonomous AI assistant infrastructure*. zeroclaw-labs/zeroclaw, GitHub. <https://github.com/zeroclaw-labs/zeroclaw>
- [6] Tran, Q.V. (2026). *GNOT: Generative Node Orchestration Technology — Reference Implementation v5.13*. gnot-io/gnot, GitHub. Apache License 2.0. <https://github.com/gnot-io/gnot>